

2.3. Les messages dans leur propre fichier

Les messages et leur place vont dans « `variables.h` » ; `main()` ne garde que les boutons.

1. Ce qu'il faut comprendre

Comme en 1.11 : chaque message devient un `Mot`, rangé dans `variables.h`. C'est l'onglet à côté de `principal.cpp`.

`principal.cpp` ne parle plus que des **boutons** : A efface `bonjour`, B le remet. Les places et les textes ne s'y écrivent plus du tout.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Mot>` → [1.7. Mot — un texte et sa place, sous un nom](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction `effacer\(\)` — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **528 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	64 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code>	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> , <code>EcrireTexte</code>	– attend de pouvoir écrire une case
EffacerCases	12 o	← <code>effacer()</code>	– efface des cases
EcrireTexte	10 o	← <code>texte()</code>	– écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← <code>bouton()</code>	– lit les huit boutons
Tuiles	176 o	← <code>CopierTuiles</code>	– 11 lettres sur 43
"BONJOUR"	7 o	← <code>texte()</code>	– un texte gravé
"APPUIE SUR A"	12 o	← <code>texte()</code>	– un texte gravé
"BONJOUR"	7 o	← <code>texte()</code>	– un texte gravé

Lettres gravées : ABEIJNOPRSU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>texte()</code>	× 3	33 o sur place, dans <code>main()</code>	+ <code>EcrireTexte</code> (10 o)
<code>bouton()</code>	× 2	10 o sur place, dans <code>main()</code>	+ <code>LireManette</code> (37 o)
<code>effacer()</code>	× 1	8 o sur place, dans <code>main()</code>	+ <code>EffacerCases</code> (12 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Mot>          // un texte et sa place, sous un seul nom
#include <texte>         // écrit un texte à l'écran
#include <bouton>        // lit un bouton de la manette
#include <effacer>       // efface des cases, ou tout le fond
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

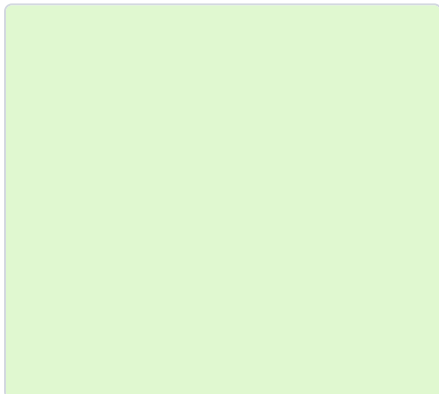
```
int main() {
    texte(bonjour);
    texte(consigne);

    while (true) {
        image();

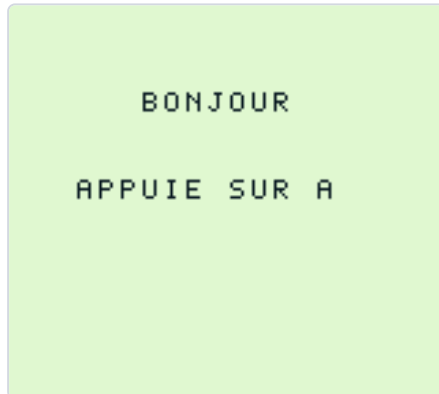
        if (bouton(A)) effacer(bonjour);
        if (bouton(B)) texte(bonjour);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

APPUIE SUR A

A — l'écran a changé

BONJOUR

APPUIE SUR A

B — l'écran a changé

5. Ce qu'on doit voir

A efface « BONJOUR » ; B le remet. La consigne reste.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ « BONJOUR » est là au départ

✓ A l'efface entièrement

✓ B le remet

✓ la consigne ne bouge pas

Ensuite : **3. Un carré de 8 × 8** — chapitre 1 : Les tout premiers pas. · [gameboy3](#) — écrire une cartouche Game Boy en C++